

NutriTrack: Sistema de Gerenciamento de Consumo Nutricional com Persistência em Arquivos Binários

Trabalho Prático — AED III (Fases 1 e 2)

Componentes do Grupo:

Pedro Augusto Silva Ferreira, Luis Felipe Almeida Rodrigues

Pontifícia Universidade Católica de Minas Gerais

Instituto de Ciências Exatas e Informática

Resumo. Este documento descreve o NutriTrack, um sistema para cadastro e gerenciamento de usuários, alimentos, refeições e registro de consumo nutricional. O sistema implementa operações CRUD completas com persistência em arquivos binários, utilizando cabeçalho com controle de ID sequencial e exclusão lógica por lápide, além de uma lista encadeada para reaproveitamento de espaço. Na Fase 2 foram acrescentados índices em *Hash Extensível* (índice primário e índice 1:N) e uma API REST com front-end SPA. A arquitetura segue o padrão MVC + DAO em Java.

1. Introdução

O acompanhamento nutricional é uma necessidade crescente. O NutriTrack foi desenvolvido para permitir que um usuário registre suas refeições e os alimentos consumidos, com informações detalhadas de macronutrientes (calorias, proteínas, carboidratos e gorduras).

O sistema foi construído em Java, seguindo o padrão MVC + DAO, e armazena todos os dados exclusivamente em arquivos binários — sem uso de banco de dados relacional. Cada arquivo possui cabeçalho com controle de ID auto-incremental e ponteiro para uma lista encadeada de registros removidos (lápide), permitindo o reaproveitamento de espaço em disco.

2. Descrição do Problema

O sistema deve permitir o cadastro e gerenciamento de **Usuários**, **Alimentos**, **Refeições** e **Consumos**, possibilitando registrar quais alimentos foram consumidos em cada refeição e em qual quantidade (em gramas). Os dados devem ser persistidos em arquivo binário com cabeçalho e controle de exclusão lógica por lápide.

3. Objetivo do Trabalho

- Desenvolver um sistema CRUD completo para as 4 entidades do domínio nutricional.
- Garantir persistência em arquivos binários (`.db`) com cabeçalho, lápide e reaproveitamento de espaço.
- Implementar relacionamentos 1:N (Usuário $\rightarrow$ Refeição) e N:N (Refeição $\leftrightarrow$ Alimento via Consumo).
- Validar chaves estrangeiras (FK) na camada Controller.
- Acelerar buscas com Hash Extensível (índice primário PK e índice 1:N FK $\rightarrow$ PK).
- Fornecer documentação com DCU, DER e Arquitetura.

4. Requisitos Funcionais

ID	Descrição
RF01	Cadastrar Usuário (nome, email, data de nascimento, telefones).
RF02	Cadastrar Alimento (nome, marca, macros por 100g, tags).
RF03	Criar Refeição vinculada a um Usuário (data, tipo, observação).
RF04	Registrar Consumo — associar Alimento a Refeição com quantidade em gramas.
RF05	Listar/consultar registros ativos (não removidos).
RF06	Excluir registros com exclusão lógica (lápide <code>'*'</code>).
RF07	Atualizar registros existentes (in-place ou realocação se maior).
RF08	Consultar registro por identificador (ID).

5. Requisitos Não Funcionais

ID	Descrição
RNF01	Interface mínima em HTML/CSS (pode ser estática).
RNF02	Persistência obrigatória em arquivos binários com cabeçalho.
RNF03	Exclusão lógica por lápide com lista de reaproveitamento.
RNF04	Documentação obrigatória (DCU + DER + Arquitetura).
RNF05	Buscas por chave em tempo médio $O(1)$ usando Hash Extensível.

6. Atores

- **Usuário:** cadastra seus dados, cria refeições e registra consumo de alimentos.
- **Administrador:** gerencia todos os cadastros (CRUD completo), incluindo exclusão e atualização.

7. Diagrama de Caso de Uso (DCU)

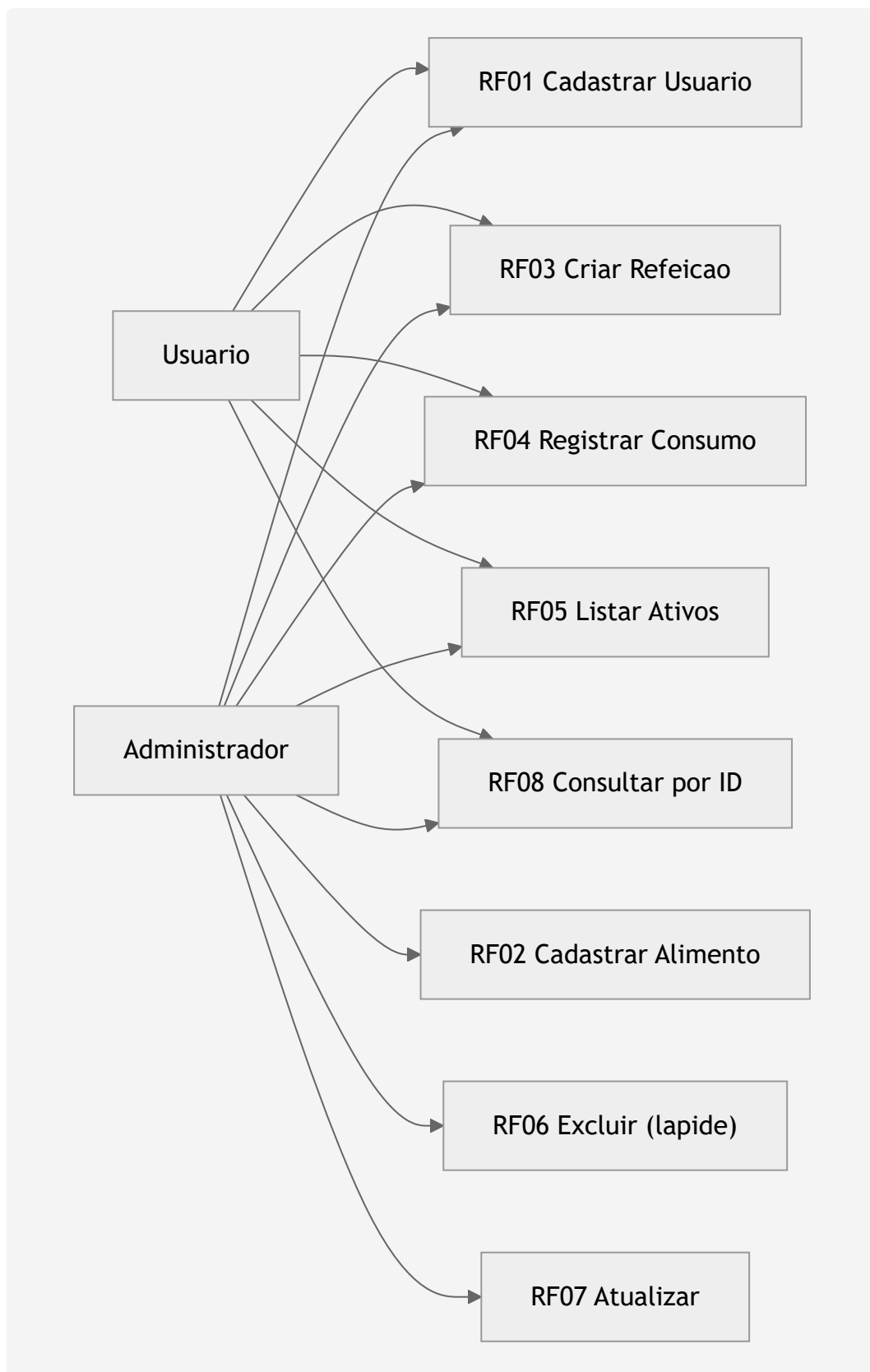


Figura 1 — Diagrama de Caso de Uso do NutriTrack.

8. Diagrama Entidade-Relacionamento (DER)

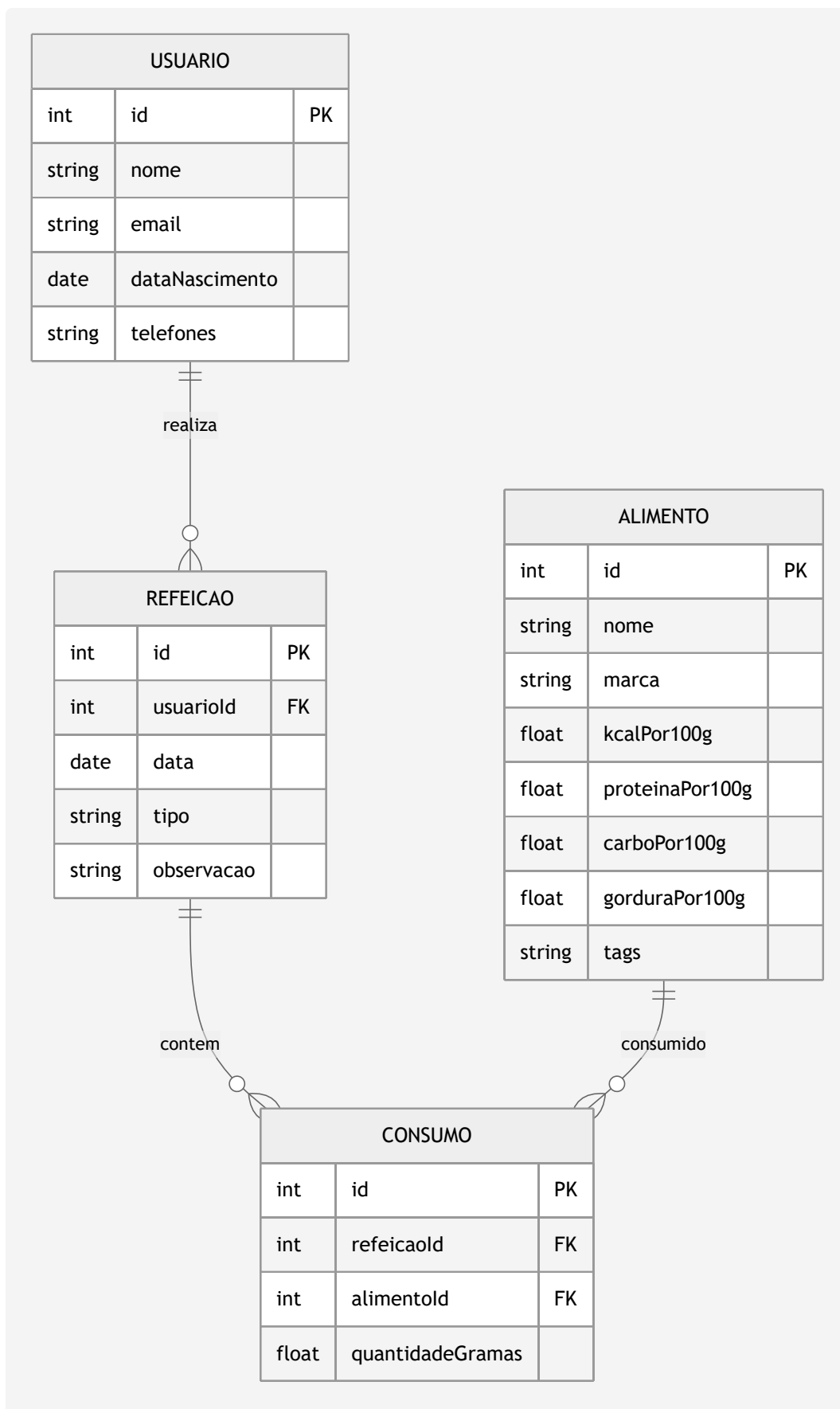


Figura 2 — Diagrama Entidade-Relacionamento do NutriTrack.

8.1 Descrição das Entidades

Usuário: Representa a pessoa que utiliza o sistema. Possui atributos simples (nome, email), um atributo do tipo data (dataNascimento, armazenado como `epochDay`) e um atributo multivalorado (telefones).

Alimento: Representa um item alimentar com informações nutricionais por 100g. Contém atributos `float` para macronutrientes e um atributo multivalorado (tags).

Refeição: Representa uma refeição realizada por um usuário em determinada data. Possui chave estrangeira `usuarioId` (1:N com Usuário) e campos tipo e observação.

Consumo: Entidade associativa que implementa o relacionamento N:N entre Refeição e Alimento, com o atributo `quantidadeGramas`.

8.2 Relacionamentos

Relacionamento	Cardinalidade	Descrição
Usuário → Refeição	1 : N	Um usuário realiza várias refeições.
Refeição → Consumo	1 : N	Uma refeição contém vários itens consumidos.
Alimento → Consumo	1 : N	Um alimento pode ser consumido em várias refeições.

9. Arquitetura Proposta

O sistema segue o padrão **MVC + DAO**, organizado em 4 camadas mais a camada de persistência genérica e os índices em Hash Extensível.

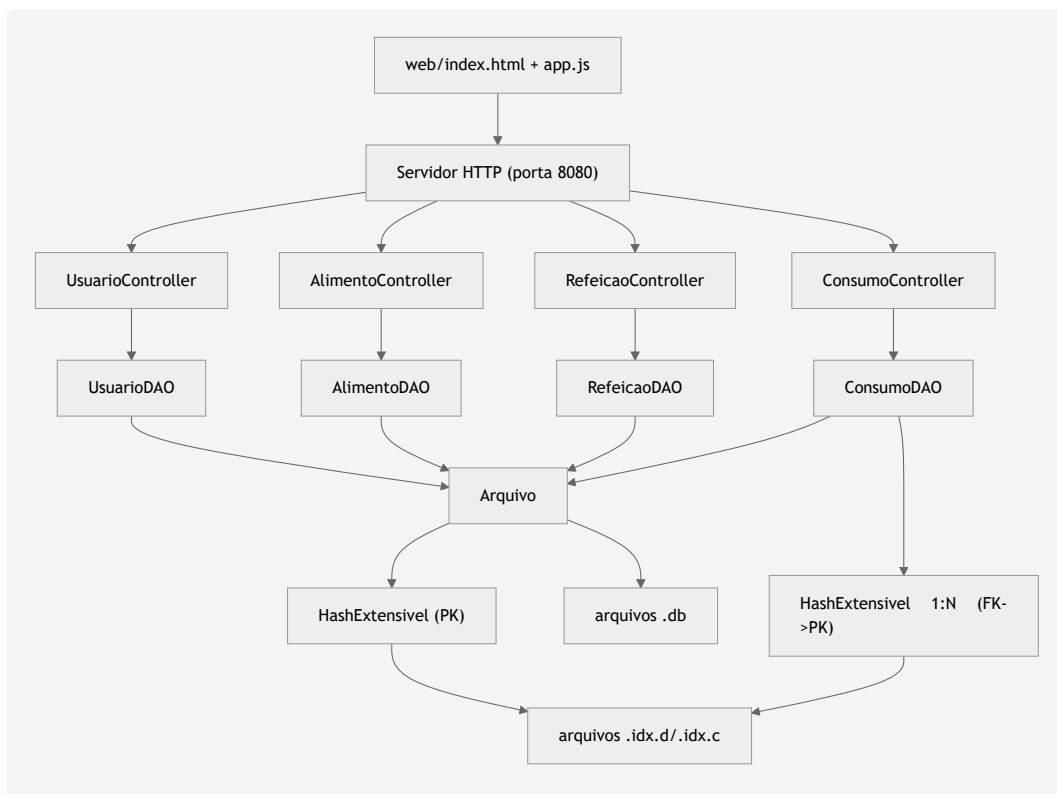


Figura 3 — Diagrama de Arquitetura em Camadas (MVC + DAO + Hash Extensível).

9.1 Descrição das Camadas

Camada	Classes	Responsabilidade
View	<code>web/index.html</code> , <code>app.js</code> , <code>styles.css</code>	SPA que consome a API REST. Possui abas para CRUD das 4 entidades.
Servidor	<code>Servidor.java</code>	Servidor HTTP embutido (<code>com.sun.net.httpserver</code>). Roteia <code>/api/*</code> para os controllers.
Controller	<code>*Controller</code>	Regras de negócio: validação de campos, validação de FKs, restrições de domínio, cascade.
Model	<code>Usuario</code> , <code>Alimento</code> , <code>Refeicao</code> , <code>Consumo</code>	Classes de domínio que implementam <code>Registro</code> .
DAO	<code>*DAO</code>	Encapsulam o acesso ao <code>Arquivo<T></code> . <code>ConsumoDAO</code> mantém também o índice 1:N.
Persistência	<code>Arquivo<T></code> + <code>HashExtensivel<T></code>	Cabeçalho, lápide, lista de removidos e índices em Hash Extensível.

10. Estrutura do Arquivo Binário

10.1 Cabeçalho (12 bytes)

Offset	Tipo	Tamanho	Descrição
0	<code>int</code>	4 bytes	Último ID atribuído (auto-incremento).
4	<code>long</code>	8 bytes	Ponteiro da cabeça da lista de registros removidos. <code>-1</code> = vazia.

10.2 Registro

Campo	Tipo	Tamanho	Descrição
Lápide	<code>byte</code>	1 byte	<code>' '</code> = ativo; <code>'*'</code> = removido.
Tamanho	<code>short</code>	2 bytes	Tamanho do array de dados em bytes.
Dados	<code>byte[]</code>	variável	Objeto serializado via <code>toByteArray()</code> .

10.3 Reaproveitamento de Espaço

Quando um registro é removido, a lápide é marcada como `'*'` e a posição é inserida em uma **lista encadeada ordenada por tamanho** (best-fit). No `create/update` que necessita de mais espaço, o sistema busca na lista um espaço removido que caiba os novos dados. Caso não exista, o registro é gravado no final do arquivo.

11. Tipos de Dados Utilizados

Tipo	Onde	Serialização
String	nome, email, marca, tipo, observação	<code>writeUTF</code> / <code>readUTF</code>
int	id, FKs (usuarioId, refeicaoId, alimentoId)	<code>writeInt</code> / <code>readInt</code>
float	kcal, proteína, carbo, gordura, gramas	<code>writeFloat</code> / <code>readFloat</code>
Data (LocalDate)	dataNascimento, data da refeição	<code>writeLong</code> (epochDay)
String multivalorada	telefones, tags	<code>writeShort(qtd) + N × writeUTF</code>

12. Hash Extensível (Fase 2)

12.1 Visão geral

Foram adicionados dois tipos de índice baseados em Hash Extensível:

1. **Índice primário** (uma por tabela): mapeia `id (PK)` → endereço no `.db`. `read`, `update` e `delete` passam a fazer um único `seek` em $O(1)$ médio.
2. **Índice 1:N**: mapeia `refeicaoId (FK)` → `consumoId (PK)`. Permite recuperar todos os consumos de uma refeição em uma única consulta hash. Também viabiliza a remoção em cascata.

12.2 Estrutura física

Arquivo	Conteúdo
<code>{nome}.idx.d.db</code>	Diretório: <code>int</code> profundidadeGlobal + $2^{\text{profGlobal}}$ ponteiros <code>long</code> .
<code>{nome}.idx.c.db</code>	Cestos: <code>byte</code> profundidadeLocal + <code>short</code> qtd + slots de tamanho fixo.

Tipo de entrada	Campos	Tamanho fixo
<code>ParIDEndereco</code> (primário)	<code>int id</code> + <code>long endereco</code>	12 bytes
<code>ParIDID (1:N)</code>	<code>int chave1 (FK)</code> + <code>int chave2 (PK)</code>	8 bytes

12.3 Operações

- **Inserção:** `idx = hashCode & (2^profGlobal - 1)`. Se o cesto enche, ocorre split com possível duplicação do diretório.
- **Leitura por chave:** 2 seeks (diretório + cesto). `read(K)` retorna *todos* os elementos com a chave (essencial para 1:N).
- **Remoção:** identifica o slot pelo `equals()`. No 1:N considera o par (FK, PK) para não remover entradas erradas.

12.4 Caminho do relacionamento 1:N



Figura 4 — Caminho de acesso ao relacionamento 1:N.

12.5 Reconstrução automática

Tanto o índice primário quanto o 1:N possuem rotina de reconstrução: se o arquivo de índice está vazio mas o `.db` de dados já contém registros, o índice é reconstruído na inicialização varrendo o arquivo de dados.

13. Front-end e API REST

O front-end é uma SPA estática em `web/` servida pelo servidor HTTP embutido em `app.Servidor` (porta 8080, sem dependências externas). A interface oferece CRUD completo das 4 entidades e demonstra o índice 1:N.

Endpoint	Método	Descrição
<code>/api/usuario</code>	GET / POST	Listar / criar usuários
<code>/api/usuario/{id}</code>	GET / PUT / DELETE	CRUD por ID (usa índice primário)
<code>/api/alimento</code>	GET / POST	Listar / criar alimentos
<code>/api/alimento/{id}</code>	GET / PUT / DELETE	CRUD por ID
<code>/api/refeicao</code>	GET / POST	Listar / criar (valida FK <code>usuarioId</code>)
<code>/api/refeicao/{id}</code>	GET / PUT / DELETE	CRUD por ID; <i>delete</i> faz cascade nos consumos
<code>/api/refeicao/{id}/consumos</code>	GET	Demonstra o índice 1:N
<code>/api/consumo</code>	GET / POST	Listar / criar (valida FKs)
<code>/api/consumo/{id}</code>	GET / PUT / DELETE	CRUD por ID

14. Validação de Entradas

Cenário	Tratamento
PK auto-incrementada	Cabeçalho controla o próximo ID; nenhum reaproveitamento.
Atualização/remoção de inexistente	<code>dao.exists(id)</code> antes da operação; lança exceção / 404.
FK inválida	Validada antes do <code>create / update</code> ; resposta 400.
Campos vazios / negativos	Validação no Controller; resposta 400.
Remoção de refeição com consumos	Cascade automático via índice 1:N.

15. Instruções de Compilação e Execução

Requisito: **JDK 17+** instalado e no PATH.

```
# Compilar
javac -d out (Get-ChildItem -Recurse src -Filter *.java | %
FullName)

# Executar servidor (recomendado)
java -cp out app.Servidor
# Abrir http://localhost:8080

# OU executar console
java -cp out app.ConsoleApp
```

16. Organização do Projeto

```
TP-AEDS-III/  
├─ dados/  
│   ├── usuario/      (usuario.db, usuario.idx.d.db,  
usuario.idx.c.db)  
│   ├── alimento/     (alimento.db + índice primário)  
│   ├── refeicao/      (refeicao.db + índice primário)  
│   ├── consumo/      (consumo.db + índice primário)  
│   └─ consumo_por_refeicao/ (índice 1:N FK→PK)  
├─ src/app/  
│   ├── Main.java, ConsoleApp.java, Servidor.java  
│   ├── Registro.java, Arquivo.java  
│   ├── model/        (Usuario, Alimento, Refeicao, Consumo)  
│   ├── controller/   (validações, FKs, cascade)  
│   └─ dao/  
│       ├── HashExtensivel.java  
│       ├── RegistroHashExtensivel.java  
│       ├── ParIDEndereco.java, ParIDID.java  
│       └─ *DAO.java  
├─ web/  
│   ├── index.html, app.js, styles.css  
│   └─ documentacao.html  
└─ README.md
```

17. Considerações Finais

O NutriTrack atende plenamente aos requisitos das duas fases. A Fase 1 entregou o CRUD completo com persistência binária, lápide e reaproveitamento de espaço. A Fase 2 acrescentou o Hash Extensível como índice primário (acelerando todas as operações por chave) e como índice 1:N (acelerando consultas de relacionamento e habilitando cascade), além de uma camada de transporte HTTP/REST e um front-end SPA. A camada de validação garante a integridade referencial entre as entidades.

18. Referências

1. ZIVIANI, N. *Projeto de Algoritmos com Implementações em Java e C++*. 3ª ed. Cengage Learning, 2010.
2. CORMEN, T. H. et al. *Introduction to Algorithms*. 4th ed. MIT Press, 2022.
3. FAGIN, R. et al. *Extendible Hashing — A Fast Access Method for Dynamic Files*. ACM TODS, 1979.

4. Oracle. *Java SE Documentation*. <https://docs.oracle.com/en/java/>. Acesso em: abr. 2026.